

Architecting Visual Multi-Agent Swarms: A Technical Blueprint for Desktop-First Orchestration, Node-Graph Canvases, and the Cursor SDK

Architectural Foundations and Swarm System Topologies

The transition from single-prompt conversational artificial intelligence to autonomous multi-agent engineering swarms requires a fundamental shift in systems architecture¹. Monolithic conversational sessions inevitably suffer from context contamination, token saturation, and non-deterministic task execution when tasked with multi-stage software delivery¹. A visual multi-agent swarm platform resolves these constraints by providing an interactive spatial canvas where users compose, configure, and inspect directed workflows of autonomous agents².

The architecture decouples high-level cognitive orchestration from low-level workspace execution². A visual canvas handles agent topology, data dependencies, and user steering². A central orchestration engine compiles the visual graph into a deterministic topological execution plan, manages asynchronous task dispatching, and serializes intermediate states to persistent storage². At the execution boundary, the system leverages an agent harness to provision isolated file workspaces, attach real-time Model Context Protocol (MCP) tool servers, run terminal commands, and stream execution telemetry back to the user interface¹.

Coordination Topology	Structural Mechanics	State & Message Routing	Failure Domain Isolation	Primary Engineering Use Cases
Topological DAG Runner [cite: 2, 4]	Directed Acyclic Graph; strict dependency tiers ⁴	Deterministic edge-based data handoffs; upstream artifacts feed downstream prompts ²	High; tasks run in parallel only when unblocked; no race conditions on shared files ²	Codebase refactoring, multi-package builds, CI/CD remediation pipelines ²
Hierarchical Supervisor	Centralized tree (Root	Planner assigns	High; workers operate in	Full-stack feature

[cite: 1, 2]	Planner to Leaf Workers) ¹	scoped manifests; workers return structured summaries ²	isolated sandboxes; supervisor aggregates progress ¹	delivery requiring spec drafting, backend, and frontend sync ¹
Decentralized Actor Mesh [cite: 1, 2]	Fully connected peer-to-peer network ¹	Asynchronous message passing via mailboxes or shared event queues ¹	Low; risk of conversational loops, token exhaustion, and conflicting workspace mutations ²	Open-ended architectural debates, red-teaming, multi-perspective code reviews ¹

For software engineering swarms, combining a topological Directed Acyclic Graph (DAG) task runner with hierarchical worker isolation provides the greatest operational reliability¹. Unstructured peer-to-peer communication channels rapidly exhaust token budgets and lead to concurrent write collisions². In contrast, modeling agent interactions as an explicit dependency graph ensures that operations such as database schema migrations, backend API scaffolding, client generation, and integration testing execute in strict chronological sequence while fanning out independent subtasks in parallel².

Visual Canvas and Node-Graph Framework Evaluation

The visual editor represents the primary operational interface. Building a fluid, Miro-like canvas requires handling arbitrary node positioning, responsive panning and zooming, custom node definitions with multiple dynamic connection ports (handles), interactive edge routing, and real-time execution feedback³.

Framework	Core Rendering Technology	Reactivity & State Integration	Custom Node Extensibility	Dynamic Handle Support	Rendering Performance Under Load	Cross-Platform Viability
React Flow (@xyflow/react) [cite: 3, 9]	DOM/SVG Hybrid with @xyflow/system [cite: 3]	First-class React Hooks; native Zustand integration	Declarative JSX components; standard HTML form	High; managed via unique IDs and useUpdateNodeInt	Optimized spatial indexing; smooth 60 FPS under 1,000+	Universal across Web, Electron, and Tauri webview

		n ⁹	inputs ³	ernals [cite: 7, 12]	nodes ³	s ¹³
Rete.js (v2)	Modular plugin architect ure (framewo rk-agnost ic)	Custom reactive event bus; adapters for React/Vu e	High; requires custom render plugins and socket abstracti ons	Moderate ; program matic socket creation	High; canvas or DOM-bas ed dependin g on render plugin	Universal across Web, Electron, and Tauri
LiteGrap h.js	HTML5 2D Canvas (Pure Canvas rendering)	Imperativ e callback graph; monolithi c canvas state	Low; manual drawing via 2D Canvas context comman ds	High program matic support; rigid visual layout	Extreme (10,000+ nodes, ComfyUI backend)	High; lacks native DOM styling for inputs/for ms
Flutter Canvas (flutter_fl ow_char t)	Skia / Impeller native rendering engine	Flutter reactive widget tree; Riverpod / Bloc	High within Dart; isolated from web ecosyste m	Moderate ; requires manual hit-testin g impleme ntations	High native GPU execution	Desktop native; requires total rewrite of web tooling

React Flow (@xyflow/react) provides the optimal foundation for this architecture³. Because agent configuration requires rich interactive controls—including multi-line system prompt textareas, searchable MCP server dropdowns, token consumption badges, and file drag-and-drop zones—rendering these nodes with native HTML and JSX avoids the substantial engineering overhead of building custom hit-testing, text selection, and form handling inside raw canvas libraries like LiteGraph³.

Dynamic handles are essential for multi-agent workflows⁷. An agent node must dynamically surface input handles for upstream agent outputs, dropped file contexts, and MCP tool connections, alongside output handles for code diffs, structured handoff summaries, and execution triggers². When handles are added or removed dynamically (for instance, when a user attaches an external database or adds a new context file), React Flow requires an explicit

call to the `useUpdateNodeInternals(nodeId)` hook⁷. This recomputes the DOM bounding client rectangles of the connection ports and recalibrates the SVG path calculation of connected edges⁷.

TypeScript

```
import React, { memo } from 'react';
import { Handle, Position, useUpdateNodeInternals } from '@xyflow/react';

interface AgentNodeData {
  label: string;
  systemPrompt: string;
  inputs: Array<{ id: string; name: string; type: string }>;
  outputs: Array<{ id: string; name: string; type: string }>;
  status: 'idle' | 'running' | 'completed' | 'failed';
}

export const AgentNode = memo(({ id, data }: { id: string; data: AgentNodeData }) => {
  const updateNodeInternals = useUpdateNodeInternals();

  React.useEffect(() => {
    updateNodeInternals(id);
  }, [id, data.inputs, data.outputs, updateNodeInternals]);

  return (
    <div className={`agent-node-card status-${data.status} p-4 rounded-xl shadow-lg border bg-slate-900 border-slate-700`} >
      <div className="font-semibold text-slate-100 mb-2 flex items-center justify-between">
        <span>{data.label}</span>
        <span className={`status-pill ${data.status}`} >{data.status}</span>
      </div>

      <div className="space-y-2 mb-4">
        {data.inputs.map(input) =>
          <div key={input.id} className="relative flex items-center text-xs text-slate-300">
            <Handle
              type="target"
              position={Position.Left}
              id={input.id}
```

```

        className="w-3 h-3 bg-blue-500 hover:scale-125 transition-transform"
      />
      <span className="ml-2">{input.name}</span>
    </div>
  ))
</div>

<div className="space-y-2">
  {data.outputs.map((output) =>
    <div key={output.id} className="relative flex items-center justify-end text-xs"
    text-slate-300>
      <span className="mr-2">{output.name}</span>
      <Handle
        type="source"
        position={Position.Right}
        id={output.id}
        className="w-3 h-3 bg-emerald-500 hover:scale-125 transition-transform"
      />
    </div>
  )}
</div>
</div>

```

To deliver modern visual feedback, edge states must reflect real-time execution³. A custom edge component tracks node execution state from a shared Zustand store¹⁰. Idle edges render with a muted gray bezier stroke (#475569, 1.5px)¹⁸. Running edges transition to a glowing cyan stroke (#06b6d4, 2.5px) utilizing an SVG animation that moves a glowing particle or gradient along the calculated path using the Web Animations API, avoiding the severe CPU and GPU performance bottlenecks associated with mutating stroke-dasharray across large diagrams⁸. Failed or blocked states transition immediately to a pulsing crimson stroke (#ef4444, 2px) indicating unfulfilled dependencies or syntax errors³.

Application Shell and Windows-First Deployment Dynamics

Developing a desktop application on Windows while ensuring clean future expansion to macOS and Linux requires selecting a shell that balances local operating system access with packaging simplicity¹.

Architectural	Electron	Tauri v2 (Rust Core)	Browser-First Web
---------------	----------	----------------------	-------------------

Parameter	(Chromium + Node.js)	+ System WebView)	App + Local Daemon
Target OS Rendering Engine	Bundled Chromium (identical across all OS platforms) ¹⁴	Windows 10/11 Evergreen WebView2 (Chromium-based) ¹⁴	Host Browser (Edge, Chrome, Firefox)
Desktop Installer Footprint	80 MB – 140 MB installer (NSIS / MSI) ²⁰	5 MB – 15 MB installer (NSIS / MSIX) ¹⁴	Zero client download (Web) + ~40 MB backend binary
Idle Memory Consumption	120 MB – 250 MB RAM ¹⁴	25 MB – 50 MB RAM ¹⁴	Browser tab overhead + server background process
Cursor SDK Runtime Integration	Native in-process execution inside Node.js background process ⁵	Requires Rust sidecar process or local Node/Bun daemon ²⁰	Executed inside backend daemon process
Windows Build Friction	Very Low; standard npm run build with electron-builder	Moderate; requires MSVC C++ Build Tools & Rust toolchain ¹⁴	Very Low; pure web stack and backend scripts
Cross-Platform Portability	Write once, package identically across Windows, macOS, Linux ¹⁴	High; requires testing against WebKitGTK on Linux ¹⁴	Universal across any operating system with a browser

Electron is the recommended runtime for the primary Windows implementation¹⁴. The primary justification centers on language runtime alignment: the official, feature-complete Cursor SDK (@cursor/sdk) is a TypeScript/Node.js package⁵. In Electron, @cursor/sdk executes directly inside a dedicated Node.js background process (UtilityProcess or Main Process) without IPC serialization friction or external interpreter dependencies⁵. If Tauri v2 were chosen, invoking the Cursor TypeScript SDK would require packaging a separate Node.js or Bun runtime as a sidecar

binary, introducing inter-process communication overhead and doubling the packaging complexity on Windows²⁰.

Operating on Windows introduces specific filesystem and process lifecycle challenges that require careful architectural planning. Unlike POSIX systems where open files and directories can be unlinked while active file descriptors exist, Windows enforces mandatory file locking²⁵. When an agent spawns an MCP server, background file watcher, or test harness whose current working directory points inside an active Git worktree, Windows locks the entire directory tree²⁶. Attempting to call `git worktree remove` throws fatal: Permission denied or leaves empty, undeletable locked directory shells on disk²⁶.

To prevent these crashes, the orchestration engine must enforce a strict, hierarchical teardown cascade:

- Terminate all active MCP child process trees using `taskkill /pid <PID> /T /F` on Windows before touching the filesystem²⁶.
- Force the host Node.js process to release active directory handles by resetting its current working directory (`process.chdir(appRoot)`) away from the worktree²⁷.
- Execute an escalating backoff retry loop (50ms, 150ms, 500ms, 2000ms) when calling `git worktree remove --force` to allow the Windows kernel time to release lingering file locks²⁷.

Additionally, interactive command-line interfaces on Windows require pseudo-terminal (PTY) allocation¹. Executing background tasks without a pseudo-terminal causes child processes to buffer output unpredictably or hang indefinitely¹. While `@cursor/sdk` bypasses CLI wrapper buffering by communicating directly with the Cursor agent bridge, any custom local tools or test verification runners executed by agents must leverage Windows ConPTY (via `node-pty`)¹. Finally, deeply nested workspace directories (such as `node_modules` inside nested agent checkouts) frequently exceed the Win32 260-character path limit²⁵. The desktop application manifest must declare `<longPathAware>true</longPathAware>`, and the internal runtime must format absolute Windows paths using the extended-length syntax prefix (`\\?\C:\path\...`) when performing directory cleanup operations²⁵.

Cursor SDK Architectural Evaluation: TypeScript vs. Python

Selecting the programming language for agent orchestration represents a foundational design choice. While Python is widely used across standalone data science workflows, analyzing runtime capabilities reveals distinct tradeoffs between the Cursor TypeScript SDK (`@cursor/sdk`) and the Cursor Python SDK (`cursor-sdk`)¹.

Architectural Feature	Cursor TypeScript SDK (<code>@cursor/sdk</code>)	Cursor Python SDK (<code>cursor-sdk</code>)
Package Maturity &	Production-ready (v1.0.31)	Early Beta (v0.1.x) tracking

Version	with rapid weekly update cycle ³¹	bridge contract updates ³²
Underlying Runtime Architecture	Native in-process Node.js harness; embeds direct agent runtime ⁵	Thin client wrapper communicating via Connect RPC to cursor-sdk-bridge [cite: 30, 33]
System Prompt Override	First-class systemPrompt option on Agent.create() and Agent.resume() [cite: 32]	Not directly exposed in base options in v0.1.x (requires prompt prefixing) ⁶
Dynamic In-Flight Steering	Supported via run.steer(text) to guide running turns in flight ³²	Not supported in v0.1.x; binary execution flow only (wait or cancel) ⁶
Execution Cancellation	Deterministic via run.cancel() across local and cloud runtimes ⁶	Supported via run.cancel() [cite: 6]
Model Catalog & Router	Full access to Composer 2.5, frontier models, and Cursor Router (auto-smart) ⁵	Full access to catalog and Cursor Router (auto-smart) ⁶
Desktop App Packaging Complexity	Zero overhead; executes directly in Electron's Node.js engine ⁵	High; requires bundling Python runtime, virtual environments, and RPC binaries ¹⁴

The TypeScript SDK (@cursor/sdk) is the definitive choice for this visual desktop application⁵. At the architectural level, both SDKs communicate with the same underlying Cursor execution harness (which handles semantic codebase indexing, AST analysis, and automated diff reconciliation)². However, the TypeScript SDK serves as the primary development platform for Cursor's engineering team, receiving features prior to other language ports⁵.

Crucially, the TypeScript SDK exposes the systemPrompt property on Agent.create() and Agent.resume()³². This allows the platform to strip standard conversational behaviors and inject machine-readable schemas and execution boundaries directly into the agent loop³². In addition, the TypeScript SDK provides run.steer(text), enabling users to inject guidance into an active agent turn without cancelling execution³².

Selecting TypeScript also enforces language uniformity across the entire application stack: the

React Flow canvas, the Zustand state store, the Electron desktop shell, the DAG orchestration engine, and the Cursor SDK all share a single TypeScript runtime³. Adopting Python would introduce substantial operational overhead, requiring the desktop application to bundle an embedded Python interpreter on Windows, manage inter-process communication serialization over localhost sockets, and maintain synchronization across three distinct runtime boundaries (Electron Node.js to Python Process to Cursor SDK Bridge)¹⁴.

TypeScript

```
import Agent, type Run from '@cursor/sdk'

export class AgentExecutionWorker {
  private activeRun: Run | null = null
  private isAborted = false

  public async executeAgentNode(
    workspacePath: string,
    systemPrompt: string,
    taskPrompt: string,
    contextData: string,
    onChunk: (text: string) => void,
    Promise: string,
  ) {
    if (this.isAborted) throw new Error('Execution aborted before launch')

    const agent = await Agent.create({
      apiKey: process.env.CURSOR_API_KEY!,
      model: id 'composer-2.5',
      systemPrompt: systemPrompt,
      local: cwd workspacePath,
    })

    try {
      const consolidatedPrompt = `### CONTEXT:\n${contextData}\n\n###
OBJECTIVE:\n${taskPrompt}`
      this.activeRun = await agent.send(consolidatedPrompt)

      for await (const event of this.activeRun.stream()) {
        if (event.type === 'assistant') {
          for (const block of event.message.content) {
            if (block.type === 'text')
```

```

        onChunk(blockText)
    }
}

const outcome = await this.activeRun.wait()
return outcome.result
finally
    await agent.Symbol.asyncDispose()
    this.activeRun = null
}

public async cancel() Promise<void>
{
    this.isAborted = true
    if (this.activeRun)
    {
        await this.activeRun.cancel()
    }
}

```

Graph Orchestration, Execution Lifecycles, and Cancellation Mechanics

Autonomous agents cannot be coordinated through unconstrained event loops without risking deadlocks or infinite execution cycles². Deterministic workflow orchestration requires compiling the visual node-graph into a topological Directed Acyclic Graph (DAG) task engine².

The orchestration engine models nodes as vertices V and handle connections as directed edges E ⁴. The engine calculates the in-degree (number of incoming dependencies) for each node using Kahn's algorithm, establishing execution tiers with computational complexity $\mathcal{O}(|V| + |E|)$ ⁴.

```

TypeScript
export interface WorkflowNode
{
    id: string
    type: 'agent' | 'fileInput' | 'mcp' | 'approval'
}

```

```

data:
  systemPrompt: string
  taskPrompt?: string
  model?: string
  workspacePath?: string
  status: 'idle' | 'queued' | 'running' | 'completed' | 'failed' | 'cancelled'
  output?: any
  key: string | any
}

```

```

export interface WorkflowEdge {
  id: string
  source: string
  sourceHandle: string
  target: string
  targetHandle: string
}

```

```

export class DAGEngine {
  private inDegrees = new Map<string, number>()
  private adjacencyList = new Map<string, string[]>()
}

```

```

  public buildSchedule(nodes: WorkflowNode[], edges: WorkflowEdge[]): string[] {
    const tiers: string[] = []
    nodes.forEach((n) => {
      this.inDegrees.set(n.id, 0)
      this.adjacencyList.set(n.id, [])
    })
  }

```

```

    edges.forEach((e) => {
      this.inDegrees.set(e.target, this.inDegrees.get(e.target)! + 1)
      this.adjacencyList.get(e.source)?.push(e.target)
    })
  }

```

```

  let currentTier = nodes.filter((n) => this.inDegrees.get(n.id) === 0).map((n) => n.id)

```

```

  while (currentTier.length > 0) {
    tiers.push(currentTier)
    const nextTier: string[] = []
  }

```

```

  for (const nodeId of currentTier) {
    for (const neighbor of this.adjacencyList.get(nodeId) || []) {

```

```

    const updatedDeg = this.inDegrees.get(neighbor) | 1 - 1
    this.inDegrees.set(neighbor, updatedDeg)
    if (updatedDeg === 0)
      nextTier.push(neighbor)
    // ...
    // ...
    currentTier = nextTier
    // ...

    const scheduledCount = tiers.reduce((acc, tier) => acc + tier.length, 0)
    if (scheduledCount > nodes.length)
      throw new Error 'Circular dependency detected in workflow graph'
    // ...

    return tiers
  }
}

```

The execution lifecycle requires strict state management within the Zustand store¹⁰. Nodes begin in an idle state. When the user initiates a run, root nodes transition to queued and immediately enter running within an asynchronous worker pool. Downstream nodes remain idle until all upstream dependency nodes reach completed status².

If an agent encounters a fatal runtime error, its status transitions to failed, halting downstream scheduling along its branch while allowing independent branches to continue execution. When a user clicks "Stop Workflow" or "Cancel Agent", the engine resolves the active Run handle and invokes `await run.cancel()`⁶. This aborts the underlying network stream, terminates local child processes, transitions the node state to cancelled, and preserves the workspace on disk for user inspection⁶.

Context Propagation, Ingestion Modalities, and MCP Architecture

Passing unedited conversational histories between connected agents creates severe token bloat and context contamination. If a downstream testing agent receives the raw, multi-turn reasoning logs, compilation traces, and failed intermediate file edits of an upstream coding agent, its reasoning capacity degrades rapidly.

To maintain clean context boundaries, the platform enforces structured artifact handoffs¹. Upstream agents synthesize their deliverables into standardized schemas before data is passed across edges². These handoffs typically include a summary of completed requirements, a list of modified files, exact interface specifications, and active blockers². Downstream agents ingest this structured summary via prompt templates that inject the upstream artifact directly into their initial context window².

Input Modality	Canvas Representation	Ingestion & Parsing Mechanism	Injected Agent Format
User File Drop [cite: 6]	Visual File Drop Node on Canvas ³	HTML5 drag-and-drop listener; copies files to project .inputs/ directory ³	Inlined as formatted Markdown code blocks or attached via SDKImage multimodal structures ⁶
Local Folder / Drive [cite: 2, 5]	Workspace Directory Node	Validated absolute Win32 path; sets local: { cwd: path } in agent config ⁵	Direct local filesystem access; indexed via Cursor semantic codebase search ²
External Database [cite: 1, 2]	Database Adapter Node	Local MCP Stdio bridge exposing dynamic SQL query tools ¹	Dynamic tool calls (query_schema, execute_read_query) inside agent loop ¹
External MCP Server [cite: 2, 6]	MCP Service Node	Stdio child process or HTTP/SSE remote client connection ⁶	Registered dynamically in Agent.create({ mcpServers }) configuration ²
Upstream Agent Output [cite: 1, 2]	Directed Canvas Edge ³	Output extraction hook parsing git diffs or JSON handoff summaries ¹	Templated context variables injected into downstream agent prompt ²

The Model Context Protocol (MCP) establishes an open standard for integrating tools and shared state¹. Instead of embedding bespoke database drivers into the orchestration engine, external databases and services run as local or remote MCP servers¹. When a user connects an MCP Node to an Agent Node, the orchestrator registers the server definition dynamically in the agent options². This allows the agent to inspect database schemas, query tables, or interact with external tracking systems using standardized tool calls without bloating the core application codebase¹.

State Persistence, Serialization, and Cross-User Portability

To ensure workflows can be saved, resumed after system reboots, and shared across teams, the platform implements a strict separation between graph topology, operational configurations, and ephemeral execution state³.

Workflows are serialized into an open JSON schema that defines viewport geometry, node configurations, system prompts, handle mappings, and environment variable requirements:

```
JSON
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "version": "1.0.0",
  "metadata": {
    "id": "wf-8f92-41a2",
    "name": "Full-Stack Feature Swarm",
    "description": "Decomposes feature specs, builds backend and frontend concurrently, runs validation.",
    "createdAt": "2026-10-03T08:00:00Z",
    "updatedAt": "2026-10-03T08:30:00Z"
  },
  "environment": {
    "requiredEnvVars": "CURSOR_API_KEY" "DATABASE_URL"
  },
  "graph": {
    "viewport": "x" 120.5 "y" 84.0 "zoom" 1.15
    "nodes": [
      {
        "id": "node-planner-1",
        "type": "agent",
        "position": "x" 100 "y" 250
        "data": {
          "label": "Systems Planner",
          "model": "composer-2.5",
          "systemPrompt": "You are a software architect. Break requirements into tasks."
          "inputs": [
            {
              "id": "in-reqs" "name" "Requirements Document" "type" "file"
            }
          ]
          "outputs":
```

```

    "id" "out-spec" "name" "Technical Plan" "type" "string"
  }
}
}
}
}
  "id" "node-backend-1"
  "type" "agent"
  "position" "x" 550 "y" 150
  "data"
  "label" "Backend Worker"
  "model" "auto-smart"
  "systemPrompt" "Implement API routes strictly following provided specifications."
  "inputs"
  "id" "in-spec" "name" "Technical Plan" "type" "string"
}
  "outputs"
  "id" "out-diff" "name" "Git Diff" "type" "diff"
}
  "mcpServers" "mcp-db-server"
}
}
}
}
  "edges"
  "id" "edge-planner-to-backend"
  "source" "node-planner-1"
  "sourceHandle" "out-spec"
  "target" "node-backend-1"
  "targetHandle" "in-spec"
  "animated" true
}
}
}
}

```

Application state is persisted locally using SQLite via `better-sqlite3`³⁶. SQLite provides durability, synchronous transaction integrity, and crash resilience without requiring an external database server¹. Write-Ahead Logging (WAL) mode is enabled to permit non-blocking concurrent reads while agent execution runs stream continuous telemetry writes³⁶:

SQL

```
PRAGMA journal_mode = WAL  
PRAGMA synchronous = NORMAL
```

```
CREATE TABLE IF NOT EXISTS workflows (  
  id TEXT PRIMARY KEY  
  name TEXT NOT NULL  
  description TEXT  
  graph_schema TEXT NOT NULL  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP  
)
```

```
CREATE TABLE IF NOT EXISTS workflow_runs (  
  id TEXT PRIMARY KEY  
  workflow_id TEXT NOT NULL  
  status TEXT NOT NULL  
  started_at DATETIME DEFAULT CURRENT_TIMESTAMP  
  ended_at DATETIME  
  FOREIGN KEY(workflow_id) REFERENCES workflows(id) ON DELETE CASCADE  
)
```

```
CREATE TABLE IF NOT EXISTS node_checkpoints (  
  id TEXT PRIMARY KEY  
  run_id TEXT NOT NULL  
  node_id TEXT NOT NULL  
  status TEXT NOT NULL  
  input_payload TEXT  
  output_result TEXT  
  token_usage_prompt INTEGER  
  token_usage_completion INTEGER  
  billed_cost_usd REAL  
  error_message TEXT  
  started_at DATETIME  
  completed_at DATETIME  
  FOREIGN KEY(run_id) REFERENCES workflow_runs(id) ON DELETE CASCADE  
)
```

When exporting workflows for external collaborators, all operational secrets (API keys, personal tokens, private database connection strings) are stripped from node configurations. The schema declares these dependencies in `environment.requiredEnvVars`. The workflow JSON is packaged into a compressed `.swarm` archive alongside referenced prompt templates and seed

assets. On import, the platform validates schema integrity, confirms graph acyclicity, prompts the importing user to supply the missing environment variables, and registers the workflow in the local database.

Phased Implementation Roadmap

To maintain engineering velocity and avoid unnecessary packaging complexity, development is structured into five sequential milestones. The project establishes visual editing and execution fundamentals locally on Windows before expanding into cross-platform distribution.

Phase	Milestone Focus	Core Deliverables	Technology Stack	Risk Mitigation Focus
Phase 1	Local Canvas & Visual Editor	Interactive node graph, custom agent cards, dynamic handles, edge routing ³	Vite, React 19, TypeScript, @xyflow/react, Tailwind CSS ³	Ensuring responsive 60 FPS panning/zooming and state binding with Zustand ³
Phase 2	Windows Desktop Shell & Isolation	Electron wrapper, secure IPC bridge, Git worktree isolation engine ¹	Electron, Node.js, better-sqlite3, simple-git ¹⁴	Windows mandatory file-locking (EBUSY) and process tree teardown ²⁶
Phase 3	Cursor SDK Execution Engine	Topological DAG scheduler, agent lifecycle management, cancellation ⁴	@cursor/sdk, Kahn's Algorithm, Web Animations API ⁴	Real-time token streaming and immediate abort handling via run.cancel() [cite: 6, 24]
Phase 4	Ingestion & MCP Integration	File drag-and-drop, workspace folder mounting,	Model Context Protocol SDK, HTML5 File API ⁶	Preventing context window saturation via structured

		dynamic MCP tool servers ²		handoff schemas ¹
Phase 5	Persistence & Distribution	SQLite checkpoints, workflow export/import, cross-platform packaging ¹⁴	better-sqlite3, electron-builder (NSIS, DMG, AppImage) ¹⁴	Secret sanitization during export; long path (MAX_PATH) handling on Windows ²⁵

Milestone 1: Local Canvas and Visual Editor

The project begins by establishing the frontend visual foundation using Vite, React, TypeScript, and Tailwind CSS. The primary deliverable is an interactive canvas utilizing @xyflow/react that supports smooth panning, zooming, minimap navigation, and background grid alignment³. Developers implement custom AgentNode components equipped with editable system prompt textareas, model selection pickers, and dynamic input/output handles managed via useUpdateNodeInternals³. State management is consolidated into a Zustand store to handle node additions, edge connections, and parameter updates cleanly¹⁰.

Milestone 2: Windows Desktop Shell and Workspace Isolation

The second phase embeds the Vite application inside an Electron desktop shell, configuring a secure Inter-Process Communication (IPC) bridge through preload.ts¹⁴. The backend introduces a dedicated Git Worktree Manager that creates isolated branch checkouts for concurrent agents, preventing staging index corruption and file write collisions¹. Defensive teardown routines are implemented to mitigate Windows EBUSY file locking: child process trees are terminated using Windows system utilities before worktree deregistration, and directory deletions execute inside an escalating backoff retry loop²⁶.

Milestone 3: Cursor SDK Execution and Topological DAG Scheduling

The third milestone integrates @cursor/sdk directly into Electron's Node.js background process⁵. The orchestration engine implements Kahn's algorithm to resolve node in-degrees and establish parallel execution tiers from visual edge connections⁴. Nodes invoke Agent.create() and dispatch tasks via agent.send(), binding execution to assigned worktree directories⁵. Execution telemetry streams back across IPC to update node status badges and trigger glowing SVG edge animations in real time⁸. The user interface connects an explicit cancellation control to run.cancel(), ensuring immediate halting of in-flight runs⁶.

Milestone 4: Multi-Modal Ingestion and MCP Tool Integration

Phase four implements multi-modal input ingestion and external tool binding⁶. An HTML5 drag-and-drop intake zone captures user files, copies them to the active project run cache, and formats them into agent context payloads or SDKImage structures⁶. Upstream agent outputs are parsed through handoff extractors that inject structured summaries into

downstream prompt templates². Dynamic MCP Node configurations are introduced, allowing agents to bind local stdio or remote HTTP/SSE Model Context Protocol servers at runtime using the `mcpServers` parameter in `Agent.create()`².

Milestone 5: Persistence, Portability, and Cross-Platform Distribution

The final milestone integrates better-sqlite3 in the Electron backend to persist workflow topologies, execution runs, and granular node checkpoints under Write-Ahead Logging (WAL) mode³⁶. The export/import module serializes graphs into standardized JSON archives, stripping sensitive credentials while prompting external users for required environment variables upon import. Packaging configurations are finalized via electron-builder to produce a signed Windows NSIS installer (.exe), with GitHub Actions workflows established to cross-compile distribution builds for macOS (.dmg) and Linux (.ApplImage)¹⁴.

Strategic Synthesis and System Conclusions

Building a visual multi-agent swarm platform transforms autonomous coding from ad-hoc script execution into a disciplined, observable software engineering system¹.

The architectural decisions reinforce one another across the entire stack:

- React Flow (@xyflow/react) provides the optimal balance of declarative HTML/JSX node customization, dynamic multi-port handle management via `useUpdateNodeInternals`, and performant SVG edge animation without the engineering friction of low-level canvas hit-testing³.
- Electron serves as the ideal application shell for Windows development while ensuring direct cross-platform portability¹⁴. It executes the official TypeScript Cursor SDK (@cursor/sdk) natively in Node.js, eliminating the dual-runtime complexity, packaging bloat, and IPC serialization bottlenecks associated with embedding an external Python environment⁵.
- The Cursor TypeScript SDK (@cursor/sdk) delivers the full feature set of the platform—including harness-level system prompt replacement, dynamic in-flight run steering, native MCP tool registration, and deterministic cancellation via `run.cancel()`⁵.
- The topological DAG orchestration engine guarantees deterministic task scheduling, while structured artifact handoffs and shared MCP backplanes prevent context window saturation across collaborative agents¹.
- Deploying defensively on Windows requires handling mandatory file locking by terminating MCP process trees prior to worktree removal, configuring extended path lengths, and implementing retry loops for filesystem teardown²⁵.

By adhering to this phased implementation strategy, engineering teams can build a visual, production-grade agent swarm platform that balances intuitive user interactions with enterprise-grade execution stability¹.

Works cited

1. Multi-Agent Swarm Architecture.pdf
2. Building Cursor Agent Swarms.pdf

3. Stop Building Node UIs from Scratch! Use xyflow Instead, <https://converter.brightcoding.dev/blog/stop-building-node-uis-from-scratch-use-xyflow-instead>
4. cursor-api · GitHub Topics, <https://github.com/topics/cursor-api?l=python>
5. Cursor SDK Tutorial: Run Coding Agents With TypeScript - DataCamp, <https://www.datacamp.com/tutorial/cursor-sdk>
6. Cursor Python SDK | Cursor Documentation, <https://cursor.com/docs/sdk/python>
7. Handles - React Flow, <https://reactflow.dev/learn/customization/handles>
8. Animating Edges - React Flow, <https://reactflow.dev/examples/edges/animating-edges>
9. reactflow - NPM, <https://www.npmjs.com/package/reactflow>
10. How to Implement an Embedded Workflow Builder that your Users, <https://www.useparagon.com/learn/how-to-implement-an-embedded-workflow-builder/>
11. Why React Flow Node-Based UI Is Better Than Other Libraries, <https://azimuahamed.medium.com/why-react-flow-node-based-ui-is-better-than-other-libraries-10-practical-reasons-2c46faaf4ae0>
12. react-flow-docs/docs/api/nodes/handle.md at main - GitHub, <https://github.com/wbkd/react-flow-docs/blob/main/docs/api/nodes/handle.md>
13. Star History Monthly Dec 2025 | React, <https://www.star-history.com/blog/react/>
14. Best Desktop App Frameworks 2026: Electron, Tauri, Neutralino, <https://www.pkgpulse.com/guides/best-desktop-app-frameworks-2026>
15. Implement a Workflow Builder with 3rd-party Actions - Paragon, <https://www.useparagon.com/learn/implement-a-workflow-builder-with-3rd-party-actions/>
16. Overview - React Flow, <https://reactflow.dev/examples>
17. xyflow/react-flow-mindmap-app: A quick and organized mind map, <https://github.com/xyflow/react-flow-mindmap-app>
18. React Flow: Everything you need to know - Synergy Codes, <https://www.synergycodes.com/blog/react-flow-everything-you-need-to-know>
19. Tuning Edge Animations in Reactflow for Optimal Performance, <https://liambx.com/blog/tuning-edge-animations-reactflow-optimal-performance>
20. Tauri vs Electron in 2025: Why We Switched and What We Learned, <https://docufiy.com/blog/tauri-vs-electron-desktop-apps-2025>
21. Build Mobile and Desktop Apps From One Codebase | Tauri 2, <https://www.mayhemcode.com/2026/07/build-mobile-and-desktop-apps-from-one.html>
22. Top 7 desktop app frameworks in 2026 | MōBrowser Blog - TeamDev, <https://teamdev.com/mobrowser/blog/top-7-desktop-app-frameworks-in-2026/>
23. GitHub - cursor/cookbook, <https://github.com/cursor/cookbook>
24. Cursor TypeScript SDK | Cursor Documentation, <https://cursor.com/docs/sdk/typescript>
25. delete worktree error · Issue #12690 · anomalyco/opencode - GitHub, <https://github.com/anomalyco/opencode/issues/12690>

26. worktree removal fails with Permission denied due to MCP ... - GitHub,
<https://github.com/anthropics/claude-code/issues/32747>
27. worktree-exit - Agent Skills Library,
<https://skills.rest/skill/worktree-exit-bbenefield89>
28. oh-my-openagent/CHANGELOG.md at dev - GitHub,
<https://github.com/code-yeongyu/oh-my-openagent/blob/dev/CHANGELOG.md>
29. cursor-sdk · GitHub Topics, <https://github.com/topics/cursor-sdk?o=asc&s=forks>
30. cursor/sdk-bridge - GitHub, <https://github.com/cursor/sdk-bridge>
31. @cursor/sdk - npm, <https://www.npmjs.com/package/%40cursor%2Fsdk>
32. SDK Changelog | Cursor Docs, <https://cursor.com/docs/sdk/changelog>
33. Cursor SDK Bridge | Cursor Documentation, <https://cursor.com/docs/sdk/bridge>
34. Cursor Router | Cursor Docs, <https://cursor.com/docs/cursor-router>
35. Using React Flow to plan a React project - LogRocket Blog,
<https://blog.logrocket.com/using-react-flow-plan-react-project/>
36. better-sqlite3 - NPM, <https://www.npmjs.com/package/better-sqlite3>
37. Experimental node-based UI for Strudel built with React Flow. · GitHub,
<https://github.com/xyflow/strudel-flow>
38. Claude Code Alternatives: 8 Tools Compared for 2026 - DataCamp,
<https://www.datacamp.com/blog/claude-code-alternatives>